

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 10 · Programación funcional: lambdas, streams, Optional y java.time

Expresiones lambda e interfaces funcionales (Predicate, Function, Consumer, Supplier)

Referencias a métodos (method references)

El API Stream: filter, map, sorted, limit, reduce, collect

Optional: orElse, orElseGet, isPresent, map

Fecha y hora con java.time: LocalDate, LocalDateTime, Period, DateTimeFormatter

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Programación funcional

Parte 1: Opción múltiple y Verdadero/Falso

4 puntos — 0,5 cada pregunta

1. ¿Cuál de estas expresiones es una lambda válida para sumar dos enteros?

- a) (a, b) -> a + b
- b) a + b -> (a, b)
- c) (a, b) => a + b
- d) int a, int b -> a + b

RESPUESTA / SOLUCIÓN

a) (a, b) -> a + b — La sintaxis de una lambda es (parámetros) -> expresión; el tipo de los parámetros se infiere.

2. ¿Qué imprime el siguiente código?

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
int r = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();
System.out.println(r);
```

- a) 6
- b) 20
- c) 55
- d) 30

RESPUESTA / SOLUCIÓN

b) 20 — Filtra los pares (2 y 4), calcula sus cuadrados (4 y 16) y los suma: 4 + 16 = 20.

3. Un stream solo puede recorrerse una vez: después de consumirlo no puede reutilizarse.

RESPUESTA / SOLUCIÓN

Verdadero — Un stream es un flujo de una sola pasada; intentar volver a operar sobre el mismo stream lanza IllegalStateException.

4. ¿Qué imprime este código?

```
Optional<String> o = Optional.empty();
System.out.println(o.orElse("vacio"));
```

- a) Optional.empty
- b) vacío
- c) null
- d) NoSuchElementException

RESPUESTA / SOLUCIÓN

b) vacío — `orElse()` devuelve el valor si está presente y, si no, el valor alternativo indicado.

5. La referencia a método `String::length` equivale a la lambda...

- a) `() -> String.length()`
- b) `s -> s.length()`
- c) `s -> String.length(s)`
- d) `String -> length`

RESPUESTA / SOLUCIÓN

b) `s -> s.length()` — Una method reference sobre un parámetro de instancia equivale a invocar el método sobre ese parámetro.

6. `Collectors.toList()` recoge los elementos de un stream en una nueva lista.

RESPUESTA / SOLUCIÓN

Verdadero — Es el collector más habitual: `stream.collect(Collectors.toList())` devuelve una `List` con los elementos resultantes.

7. ¿Qué fecha imprime este código?

```
LocalDate hoy = LocalDate.of(2026, 3, 10);  
System.out.println(hoy.plusDays(20));
```

- a) 2026-03-30
- b) 2026-03-20
- c) 2026-04-01
- d) 2026-04-30

RESPUESTA / SOLUCIÓN

a) 2026-03-30 — `LocalDate` es inmutable; `plusDays(20)` devuelve una nueva fecha: 10 de marzo + 20 días = 30 de marzo.

8. ¿Qué interfaz funcional se usa para representar una condición que devuelve `true` o `false` sobre un objeto?

- a) `Consumer<T>`
- b) `Supplier<T>`
- c) `Predicate<T>`
- d) `Function<T, R>`

RESPUESTA / SOLUCIÓN

c) `Predicate<T>` — Su método `test(T)` devuelve `boolean`; es la interfaz típica para `filter()`.

Parte 2: Análisis y depuración

3 puntos

1. El siguiente programa tiene 2 errores. Encuéntralos, explica por qué fallan y propón la corrección. (1,5 pts)

```
import java.util.*;
import java.util.stream.*;
public class StreamBug {
    public static void main(String[] args) {
        List<String> nombres = List.of("Ana", "Bea", "Carlos");
        Stream<String> st = nombres.stream();
        st.forEach(System.out::println);
        st.filter(s -> s.length() > 2).forEach(System.out::println);

        int limite = 2;
        nombres.stream().filter(s -> s.length() >= limite);
        limite = 3;
    }
}
```

RESPUESTA / SOLUCIÓN

1. Se usa el mismo stream `st` dos veces: tras `forEach()` el stream está consumido y la segunda operación lanza `IllegalStateException` (stream has already been operated upon). Corrección: crear un stream nuevo por operación: `nombres.stream().filter(...)`...
2. La variable `limite` se usa dentro de una lambda y después se reasigna (`limite = 3`): las lambdas solo pueden capturar variables effectively final → error de compilación. Corrección: no reasignarla o usar otra variable.

-
2. Analiza el siguiente pipeline y determina qué nombres imprime y en qué orden (traza manual). (1,5 pts)

```
List<String> nombres = List.of("Ana", "Bea", "Carlos", "David", "Eva");
nombres.stream()
    .filter(s -> s.length() >= 3)
    .sorted()
    .forEach(System.out::println);
```

RESPUESTA / SOLUCIÓN

1. `filter(s -> s.length() >= 3)`: todas las cadenas tienen 3 o más letras, así que ninguna se descarta (Eva tiene 3).
 2. `sorted()` ordena alfabéticamente (orden natural de `String`).
- Salida: Ana, Bea, Carlos, David, Eva.
-

Parte 3: Ejercicios de programación

8 puntos

1. Con streams: dada la lista [1, 2, 3, 4, 5, 6, 7, 8], calcula la suma de los cuadrados de los números pares y muestra los 3 mayores pares en orden descendente. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;
import java.util.stream.*;

public class StreamsNumeros {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6, 7, 8);
        int sumaCuadradosPares = numeros.stream()
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n * n)
            .sum();

        System.out.println("Suma de cuadrados de pares: " + sumaCuadradosPares);
        List<Integer> paresDesc = numeros.stream()
            .filter(n -> n % 2 == 0)
            .sorted(Comparator.reverseOrder())
            .limit(3)
            .collect(Collectors.toList());

        System.out.println("Tres mayores pares: " + paresDesc);
    }
}
```

Salida: Suma de cuadrados de pares: 120 (4+16+36+64) y Tres mayores pares: [8, 6, 4].

2. Con una lista de alumnos (nombre, nota), usa streams para: (a) calcular la nota media, (b) mostrar los aprobados y (c) mostrar el alumno con mejor nota. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class StreamsAlumnos {

    static class Alumno {

        String nombre; double nota;

        Alumno(String nombre, double nota) { this.nombre = nombre; this.nota = nota; }

        public String toString() { return nombre; }

    }

    public static void main(String[] args) {

        List<Alumno> alumnos = List.of(

            new Alumno("Ana", 7.5), new Alumno("Luis", 4.9),

            new Alumno("Eva", 9.2), new Alumno("Juan", 5.5));

        double media = alumnos.stream()

            .mapToDouble(a -> a.nota)

            .average()

            .orElse(0);

        System.out.println("Nota media: " + media);

        System.out.println("Aprobados:");

        alumnos.stream()

            .filter(a -> a.nota >= 5)

            .forEach(System.out::println);

        Alumno mejor = alumnos.stream()

            .max(Comparator.comparingDouble(a -> a.nota))

            .orElse(null);

        System.out.println("Mejor: " + mejor);

    }

}
```

3. **Implementa un método que devuelva `Optional<Alumno>` al buscar un alumno por nombre en una lista (sin distinguir mayúsculas). En el main usa `orElse()` y `ifPresentOrElse()` para gestionar la ausencia. (2 ptos)**
- Salida: Nota media: 6.775; aprobados Ana, Eva y Juan; Mejor: Eva.

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class BuscarAlumno {

    static class Alumno {

        String nombre; double nota;

        Alumno(String nombre, double nota) { this.nombre = nombre; this.nota = nota; }

        public String toString() { return nombre + " (" + nota + ")"; }

    }

    static Optional<Alumno> buscar(List<Alumno> lista, String nombre) {

        for (Alumno a : lista) {

            if (a.nombre.equalsIgnoreCase(nombre)) return Optional.of(a);

        }

        return Optional.empty();

    }

    public static void main(String[] args) {

        List<Alumno> alumnos = List.of(

            new Alumno("Ana", 7.5), new Alumno("Eva", 9.2));

        Alumno encontrado = buscar(alumnos, "eva")

            .orElse(new Alumno("Desconocido", 0));

        System.out.println("Resultado: " + encontrado);

        buscar(alumnos, "Pedro").ifPresentOrElse(

            a -> System.out.println("Nota de " + a.nombre + ": " + a.nota),

            () -> System.out.println("Pedro no esta en la lista"));

    }

}
```

4. Con `java.time` pide una fecha de nacimiento (AAAA-MM-DD), calcula la edad con `Period.between`, los días vividos con `ChronoUnit.DAYS.between` y muestra la fecha de hoy formateada como `dd/MM/yyyy`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.Scanner;

public class Fechas {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Fecha de nacimiento (AAAA-MM-DD): ");
        LocalDate nacimiento = LocalDate.parse(sc.nextLine());
        LocalDate hoy = LocalDate.now();
        Period periodo = Period.between(nacimiento, hoy);
        System.out.println("Edad: " + periodo.getYears() + " años, "
            + periodo.getMonths() + " meses y " + periodo.getDays() + " días");
        long diasVividos = ChronoUnit.DAYS.between(nacimiento, hoy);
        System.out.println("Días vividos: " + diasVividos);

        DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        System.out.println("Hoy: " + hoy.format(formato));

    }

}
```

Parte 4: Pregunta teórica

1 punto

1. Explica qué es una expresión lambda y qué es una interfaz funcional. ¿Qué ventajas aportan los streams frente a los bucles tradicionales? Pon un ejemplo comparativo.

RESPUESTA / SOLUCIÓN

Una lambda es una función anónima: (parámetros) -> expresión, que puede asignarse a una interfaz funcional. Una interfaz funcional tiene un único método abstracto (Predicate, Function, Consumer, Supplier, Comparator...).

Ventajas de los streams: código más conciso y legible, operaciones encadenadas (filter, map, sorted, limit...), evaluación perezosa (las operaciones intermedias solo se ejecutan si hay una operación final) y facilidad para paralelizar con parallelStream().

Ejemplo: filtrar y mostrar nombres que empiezan por 'A'.

Bucle tradicional:

```
for (String s : nombres) if (s.startsWith("A")) System.out.println(s);
```

Con streams:

```
nombres.stream().filter(s -> s.startsWith("A")).forEach(System.out::println);
```



Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)